

DYNAMIC PROGRAMMING - Sprint 4

Integrantes:

-Lucca Borges RM554608

-Ruan Vieira RM557599

-Rodrigo Carnevale RM558148

Sumário

1. Introdução
2. Modelagem do Problema
3. Estrutura de Estados e Lead Time
4. Programação Dinâmica (PD)
5. Visualização e Interpretação
6. Simulação da Política Ótima
7. Análise de Sensibilidade
8. Teste de Desempenho
9. Estruturas de Dados Utilizadas
10. Lógica de Cálculo e Transição
11. Conclusão

1. Introdução

O presente relatório tem como objetivo descrever as estruturas e algoritmos utilizados no desenvolvimento de um modelo de controle de estoque baseado em Programação Dinâmica. O projeto faz parte da Sprint 4, intitulada DYNAMIC PROGRAMMING, e tem como foco a minimização do custo total esperado de operação em um sistema com demanda estocástica e horizonte de tempo finito.

2. Modelagem do Problema

A modelagem envolve um ambiente onde a demanda diária segue uma distribuição de Poisson, permitindo representar a incerteza típica de processos de consumo. O sistema possui parâmetros como capacidade máxima de estoque, custo fixo de pedido, custo unitário, custo de estocagem e penalidade por falta de produto. A variável de decisão é a quantidade a ser pedida em cada período.

Para manter o código organizado, foi criada uma estrutura de dados central que agrupa todos os parâmetros, permitindo alterar cenários de forma simples. Além disso, o modelo considera a possibilidade de 'lead time', ou seja, o tempo entre a realização do pedido e a chegada dos itens.

3. Estrutura de Estados e Lead Time

O estado do sistema é representado pelo nível atual de estoque e, quando há lead time, também por um pipeline de pedidos pendentes. Essa estrutura possibilita calcular o estoque disponível em cada etapa e prever futuras chegadas de produtos. A cada nova decisão, o pipeline é atualizado para refletir a entrada de novos pedidos e a entrega dos antigos.

4. Programação Dinâmica (PD)

A Programação Dinâmica foi utilizada para determinar a política ótima de pedidos que minimiza o custo total esperado. O modelo trabalha com uma função valor, que associa a cada estado e instante de tempo o menor custo possível até o final do horizonte de planejamento.

Foram implementadas duas versões do algoritmo: uma recursiva, que utiliza memoização para evitar recomputações, e uma iterativa, que percorre o horizonte de tempo de forma reversa (do último período para o primeiro). A equivalência entre as duas abordagens foi validada para garantir consistência nos resultados.

5. Visualização e Interpretação

Para interpretar os resultados, foi gerado um mapa de calor que mostra, para cada combinação de tempo e nível de estoque, a quantidade ótima de pedido. Essa visualização

auxilia na compreensão da estratégia ideal e na identificação das regiões onde é vantajoso realizar novos pedidos.

6. Simulação da Política Ótima

Com base na política ótima obtida pela Programação Dinâmica, foi possível simular o comportamento do sistema sob diferentes cenários de demanda. A simulação permitiu avaliar o desempenho prático da estratégia, estimando custos médios, níveis de serviço e a frequência de faltas de estoque.

7. Análise de Sensibilidade

A análise de sensibilidade foi conduzida para entender o impacto das variações dos parâmetros do modelo, como o custo fixo de pedido (K) e a penalidade por falta (p). Os resultados mostraram como pequenas alterações nesses parâmetros podem influenciar de forma significativa o custo total e a política ótima de pedidos.

8. Teste de Desempenho

Para complementar a análise, foi realizado um teste de desempenho utilizando a biblioteca `time` para comparar o tempo de execução das versões recursiva e iterativa da Programação Dinâmica.

Os resultados mostraram que ambos os métodos produzem resultados idênticos, validando a consistência da implementação, porém com tempos de execução distintos:

Tempo (recursiva): 0.6225 s

Tempo (iterativa): 0.4438 s

A versão iterativa apresentou desempenho superior, pois evita chamadas recursivas e aproveita melhor a estrutura de laços e vetores. Ainda assim, a versão recursiva é útil para fins de clareza e depuração, especialmente quando combinada com memoização (`@lru_cache`).

9. Estruturas de Dados Utilizadas

Foram empregadas estruturas de dados eficientes para lidar com o grande número de estados e decisões. Quando não há lead time, são utilizados arrays NumPy, que oferecem acesso rápido e eficiente. Já em situações com lead time, o estado inclui tuplas de pipeline e é armazenado em dicionários, que permitem indexação flexível. Também foram utilizadas

tuplas imutáveis para possibilitar o uso de cache e a função de memoização (`@lru_cache`) para otimizar o desempenho computacional.

10. Lógica de Cálculo e Transição

Em cada etapa da Programação Dinâmica, para cada estado (estoque e pipeline) e ação possível (quantidade pedida), o algoritmo calcula o custo total esperado. Esse custo inclui o custo do pedido, o custo de armazenagem e as penalidades por falta de produto. Após considerar todas as demandas possíveis, ponderadas pela probabilidade da distribuição de Poisson, a ação com menor custo esperado é selecionada como ótima.

11. Conclusão

O uso da Programação Dinâmica mostrou-se uma abordagem eficaz para resolver problemas de controle de estoque com incerteza na demanda. O modelo desenvolvido permite tomar decisões ótimas com base em dados probabilísticos, garantindo eficiência operacional e redução de custos. Além disso, a análise de sensibilidade e as simulações foram essenciais para validar o modelo e compreender melhor seu comportamento sob diferentes condições.